

Image Processing Lab

Signals & Systems, built from scratch in Python

Zarif Ishmam | 2305035

Zarif Mahir | 2305032

September 26, 2026

CSE220 Signals and Systems Sessional Course
Department of CSE, BUET

What is Image Processing Lab?

A web app that edits images with **hand-written** Signals & Systems algorithms.

- Pick a photo or use the camera
- Choose one of **11 operations**
- See our result beside a library's (Pillow, SciPy, numpy.fft), both timed

The rule: no library FFT or convolution. NumPy is used for array arithmetic only.

 **IMAGE PROCESSING**
LABORATORY

SOURCE

No file chosen

+

Use camera



OPERATION

▼

Blur

convolution

Sharpen

cft high-pass

Edges

cft high-pass

Denoise

fft notch

Compress

fft top-k

Encrypt

fft phase key

Decrypt

key + saved png

Brightness

point op

Channels

rgb offsets

Filters

per pixel

Resize

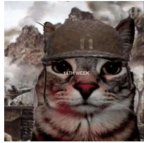
nearest pixel

NOISE

DENOISE · PERIODIC NOISE AT 0.3 · NOTCH, 6 PEAKS FOUND, WIDTH 3 · 99.8% OF THE SPECTRUM KEPT ·

256 x 256 · 0.83 s

ORIGINAL



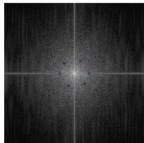
NOISY

24.5 dB



SPECTRUM

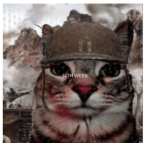
log |X|, centred - 256 x 256



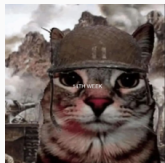
DENOISED

37.8 dB

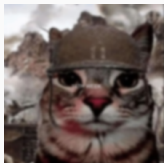
SAVE PNG



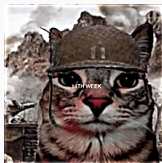
What it can do



Input
your photo



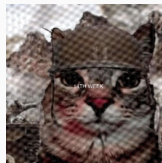
Blur
convolution



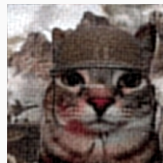
Sharpen
CFT high-pass



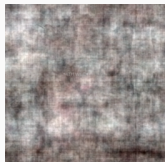
Edges
CFT high-pass



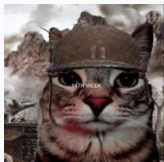
Denoise
FFT notch filter



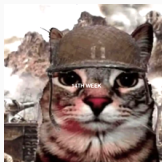
Compress
FFT top- k



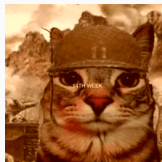
Encrypt
random phase



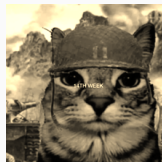
Decrypt
inverse phase



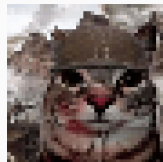
Brightness
gain



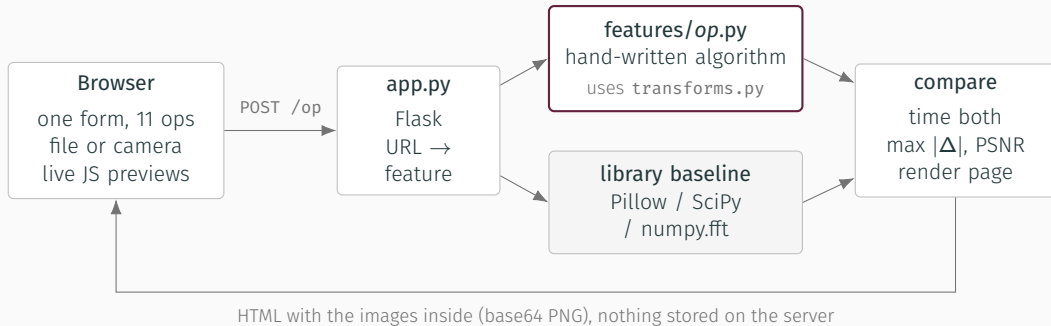
Channels
RGB offsets



Filters
per-pixel maps



Resize
sampling



Python 3.14 · Flask · NumPy (arithmetic only) · Pillow, SciPy (baselines only) · vanilla JS

Each output pixel is a weighted average of its $k \times k$ neighbourhood. The weights, the **kernel**, decide the kind of blur.



input



box



Gaussian



disc (lens)



motion 30°

9 × 9 kernels · vs Pillow / SciPy: $\max |\Delta| \leq 1$ · animated live in the browser

Signals & Systems · LTI system · impulse response · 2D convolution

Kernel h = impulse response:

$$y[m, n] = \sum_k \sum_l h[k, l] x[m - k, n - l]$$

- Same weights everywhere:
shift-invariant
- $\sum h = 1 \Rightarrow H(0, 0) = 1$ (DC kept)
- Averaging $\Rightarrow$ low-pass filter

Gaussian: $h \propto e^{-(k^2+l^2)/2\sigma^2}$, $\sigma = \text{size}/6$

Edge pixels replicated at the border.

Cost $O(MNk^2)$: 11×11 at 500 px ≈ 10 s.

```
def convolve2d(x, h):  
    h = h[::-1, ::-1] # flip = convolution  
    p = pad(x, K // 2, "edge") # replicate border  
    for m in range(H):  
        for n in range(W):  
            s = 0.0  
            for k in range(K):  
                for l in range(K):  
                    s += p[m+k, n+l] * h[k, l]  
            y[m, n] = s  
    return y # per colour channel  
  
def gaussian_kernel(size):  
    sigma = size / 6  
    d = arange(size) - size // 2  
    g = exp(-(d[:,None]**2 + d[None,:]**2)  
            / (2 * sigma**2))  
    return g / g.sum() # sum(h) = 1
```

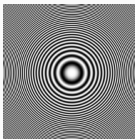
Every new pixel copies the old pixel under its centre (nearest neighbour). Nothing is averaged.



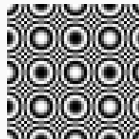
input



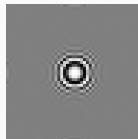
$\times 0.2$, then $\times 5$



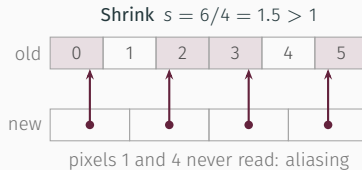
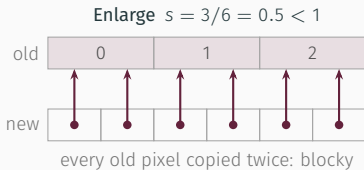
zone plate



$\downarrow 4$, nearest (ours)



$\downarrow 4$, low-pass first



blocky when enlarged, false rings when shrunk · identical to Pillow's NEAREST: $\max |\Delta| = 0$

Signals & Systems · sampling · Nyquist · aliasing · zero-order hold

Resizing resamples the image on a new grid, step $s = \text{old}/\text{new}$:

$$y[i] = x[\lfloor (i + \frac{1}{2})s \rfloor]$$

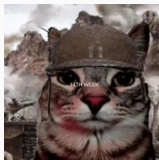
- **Shrink** ($s > 1$): frequencies above the new Nyquist $\frac{1}{2s}$ fold back:
aliasing
- Cure: low-pass *before* sampling
- **Enlarge** ($s < 1$): each sample held, a zero-order hold: blocky

```
def old_positions(old, new):
    step = old / new
    pos = step / 2          # centre of pixel 0
    out = []
    for i in range(new):
        out.append(int(pos)) # old pixel below
        pos += step          # summed, as Pillow
    return out

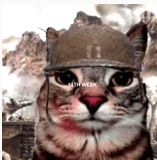
def resize_nearest(img, scale):
    rows = old_positions(H, round(H * scale))
    cols = old_positions(W, round(W * scale))
    for y in range(len(rows)):
        for x in range(len(cols)):
            out[y, x] = img[rows[y], cols[x]]
    return out
```

Brightness, Channels & Filters

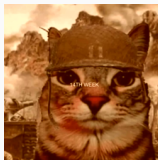
Each output pixel depends only on the same input pixel, so the browser previews these **live** while a slider moves.



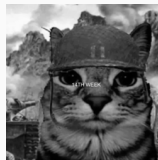
input



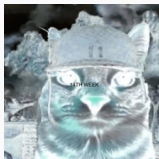
brightness $\times 1.4$



R +50, B -50



grayscale



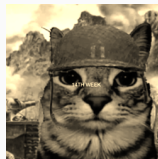
invert



black & white



pixelate



vintage

Signals & Systems system properties · memoryless · linear vs non-linear

Linear: $T\{ax_1 + bx_2\} = aT\{x_1\} + bT\{x_2\}$

System	y	memoryless	linear
Brightness	gx	✓	✓
Channels	$x + d_c$	✓	affine
Gray, sepia	Mx	✓	✓
Invert	$255 - x$	✓	affine
Black & white	$255 [luma \geq T]$	✓	×
Pixelate	tile mean	×	✓

Clipping to $[0, 255]$ afterwards is itself a non-linear system (saturation).

```
for y in range(H):           # each pixel
    for x in range(W):       # on its own
        out[y, x] = f(img[y, x])

bright = lambda p: gain * p
shift  = lambda p: p + (dR, dG, dB)
gray   = lambda p: p + a*(luma(p) - p)
invert = lambda p: p + a*(255 - 2*p)
bw      = lambda p: 255 * (luma(p) >= T)

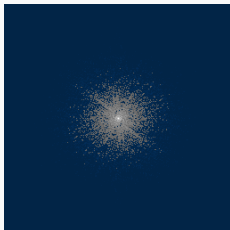
out = clip(out, 0, 255).astype(uint8)
```

Compress

Keep only the strongest 5% of the Fourier coefficients, store them compactly, rebuild the image from them.



original 180.8 KB raw



bins kept



ours 22.4 KB 26.1 dB



JPEG 21.5 KB 42.9 dB

8.1× smaller than raw · at the same size JPEG wins by 17 dB: it also quantises and entropy-codes

Signals & Systems · energy compaction · Parseval · conjugate symmetry · linearity

Energy is the same in both domains
(Parseval):

$$\sum_{m,n} |x[m,n]|^2 = \frac{1}{MN} \sum_{u,v} |X[u,v]|^2$$

A photo's energy sits in few coefficients,
so dropping the smallest $|X|$ costs little.

Real image $\Rightarrow$ **conjugate symmetry**, so
one bin of each pair is enough:

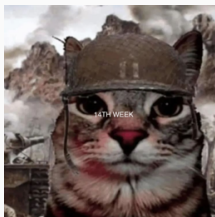
$$X[-u, -v] = X^*[u, v]$$

Bins picked once, on luma (linearity). 14 bytes
per stored pair.

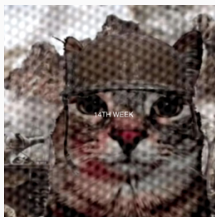
```
def encode(img, keep):
    X = [fft2(pad(img[c])) for c in RGB]
    L = .299*X[0] + .587*X[1] + .114*X[2]
    top = argpartition(abs(L), -k)[-k:] # top k
    top = top | partner(top)          # whole pairs
    idx = one bin of each pair
    vals = float16(X[:, idx] / (M*N)) # 2 B
    return uint16(idx), vals

def decode(idx, vals):
    X = zeros((3, M, N), complex)
    X[:, idx] = vals * M*N
    X[:, partner(idx)] = conj(vals * M*N)
    return [ifft2(X[c]).real for c in RGB]
```

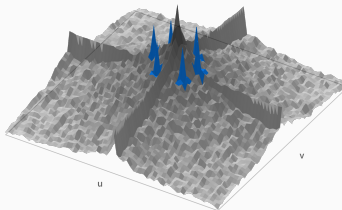
Stir periodic stripes into the photo, find them in the spectrum, cut them out.



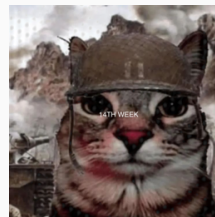
clean



noisy 24.5 dB



spectrum in 3D, 6 spikes notched



denoised 37.8 dB

app defaults (stripes 30%, notch width 3) · 99.8% of the spectrum kept · vs numpy.fft: $\max |\Delta| = 0$

The FFT, from scratch

Signals & Systems · DFT · Cooley–Tukey divide & conquer · $O(N \log N)$

The DFT of N samples:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad W_N = e^{-j2\pi/N}$$

Direct: $O(N^2)$. Split even (E) and odd (O):

$$X[k] = E[k] + W_N^k O[k]$$

$$X[k + \frac{N}{2}] = E[k] - W_N^k O[k]$$

$\log_2 N$ stages of butterflies: $O(N \log N)$.

$N = 1024$: DFT 148 ms $\rightarrow$ FFT 1.2 ms, equal to numpy.fft within 10^{-12} .

Sides reflect-padded to 2^k : a zero-padded edge is a step, which is broadband.

```
def fft(x):                                # N = 2^m
    X = bit_reverse_order(x)
    for s in range(1, m + 1):              # log2 N stages
        M = 2**s
        wM = exp(-2j * pi / M)             # once per stage
        for l in range(0, N, M):           # each block
            w = 1
            for k in range(M // 2):         # butterflies
                a, b = l + k, l + k + M//2
                g, h = X[a], w * X[b]
                X[a], X[b] = g + h, g - h
            w *= wM
    return X

ifft = lambda X: conj(fft(conj(X))) / N    # reuse

fft2 = lambda a: fft_columns(fft_rows(a)) # separable
```

Signals & Systems frequency response · notch filter · convolution theorem

A sinusoid is **two spikes** in the spectrum:

$$\cos(2\pi k_0 \cdot r) \longleftrightarrow \frac{1}{2} [\delta(k - k_0) + \delta(k + k_0)]$$

Filtering is multiplication in frequency:

$$Y[k] = H[k] X[k] \iff y = h * x$$

Stripes sit far from the centre, the photo near it. **Notch** H : a small Gaussian hole at each spike.

New here: the spectrum is centred (shift), so low frequencies sit in the middle.

```
X = shift(fft2(pad_reflect(luma(noisy))))
A = abs(X)
floor = 4 * quantile(A[outside centre], 0.999)

H = ones_like(A)
while A.max() > floor:           # hunt the spikes
    p = argmax(A)
    H *= 1 - exp(-dist(p)**2 / (2 * s**2))
    A[near p] = 0                # look for the next

for c in (R, G, B):             # same mask
    Xc = shift(fft2(pad_reflect(x[c])))
    y[c] = ifft2(unshift(Xc * H)).real[:h, :w]
```

Sharpen & Edges

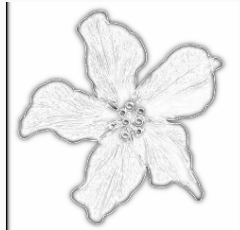
Keep only the part of the image that changes fast. Add it back: **sharper**. Keep only its size: **edges**.



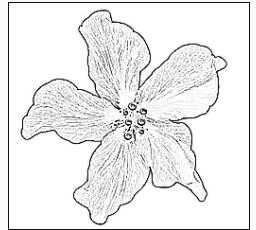
before



sharpened



our edges



Pillow FIND_EDGES

cutoff 15, amount 1.5 · Pillow uses a different method (spatial kernels), so the two are compared by eye

Signals & Systems continuous Fourier transform · numerical integration · high-pass filter

Not the DFT here: the **continuous** FT of $f(x, y)$ on $[-1, 1]^2$:

$$F(u, v) = \iint f(x, y) e^{-j2\pi(ux+vy)} dx dy$$

evaluated by the **trapezoidal rule**, along x then y .

Gaussian **high-pass** (D = distance from DC):

$$H(u, v) = 1 - e^{-D^2/2c^2}$$

With $g = \mathcal{F}^{-1}\{HF\}$, the fine detail:

$$\text{sharpen} = f + \alpha g \quad \text{edges} = |g|$$

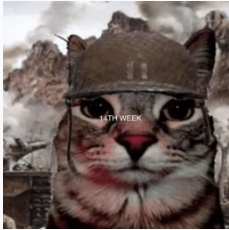
```
def cft(f, x, y, u, v):          # trapezoid rule
    for j in range(len(u)):      # along x, each u
        t = 2*pi*u[j]*x
        r1[:, j] = trapz(f * cos(t), x)
        r2[:, j] = trapz(f * sin(t), x)
    for i in range(len(v)):      # along y, each v
        c, s = cos(2*pi*v[i]*y), sin(2*pi*v[i]*y)
        re[i] = trapz(r1*c - r2*s, y)
        im[i] = -trapz(r2*c + r1*s, y)
    return re, im

D = sqrt((u/du)**2 + (v/dv)**2) # distance from DC
H = 1 - exp(-D**2 / (2 * c**2)) # Gaussian high-pass
detail = icft(re * H, im * H)

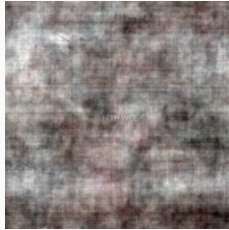
sharpened = f + amount * detail
edges = 1 - abs(detail) / abs(detail).max()
```

Encrypt & Decrypt

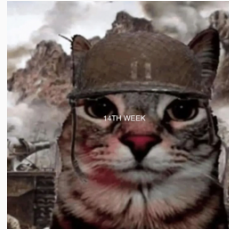
Scramble the phase of the spectrum with a passphrase. The same passphrase unscrambles it; any other gives noise.



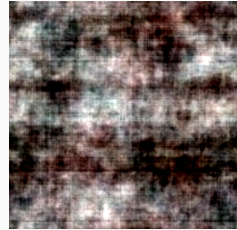
original



ciphertext (a PNG)



key cse220 53.7 dB



key cse221 9.6 dB

the PNG carries the scheme, stretch and crop inside it, so a saved ciphertext opens later with only the key

Signals & Systems · all-pass system · phase vs magnitude · invertible systems

Multiply the spectrum by a unit-magnitude mask:

$$E = \mathcal{F}^{-1}\{\mathcal{F}\{x\} H\}, \quad H = e^{j\varphi}, \quad |H| = 1$$

- **All-pass**: energy unchanged
- Phase holds structure: scrambled
- $H^{-1} = H^* = e^{-j\varphi}$: **exact** recovery
- Wrong key: $e^{j(\varphi - \varphi')}$, another scramble

Odd phase, $\varphi[-u, -v] = -\varphi[u, v]$, keeps E real, so it saves as a PNG.

```
def phase_mask(key, shape):
    r = rng(sha256(key)).random(shape)
    phi = pi * (r - mirror(r))    # odd phase
    return exp(1j * phi)         # |H| = 1

def encrypt(x, key):
    H = phase_mask(key, x.shape)
    E = ifft2(fft2(x) * H).real   # stays real
    return to_png(E)              # + scheme, crop

def decrypt(E, key):
    H = phase_mask(key, E.shape)
    return ifft2(fft2(E) * conj(H)).real
```

Results: ours vs the library

Operation	Baseline	Size	Ours	Baseline	max $ \Delta $
FFT, $N = 1024$	direct DFT, numpy.fft	1024	1.2 ms	148 ms	10^{-12}
Blur, box 5×5	Pillow BoxBlur	500×500	2.05 s	1.1 ms	1
Denoise, notch	numpy.fft	256×256	0.83 s	2.5 ms	0
Encrypt, phase	numpy.fft	256×256	0.71 s	3.7 ms	0
Brightness $\times 1.4$	ImageEnhance	500×500	60 ms	0.26 ms	1
Channels	Image.point (LUT)	500×500	60 ms	0.26 ms	0
Grayscale	Image.blend	500×500	208 ms	0.14 ms	1
Resize 50%	resize(NEAREST)	500×500	8.5 ms	0.04 ms	0

Same answers (within 1 grey level: rounding vs truncation), 190–1900× slower: pure-Python loops against C.

Sharpen, edges and compress use a different method from their baselines, so they are compared by eye and PSNR instead.

Thank you

Questions?